



GNU make

David Darras

Date de création : 14/09/2024
Dernière modification : 29/12/2024

1 Introduction à make

- La programmation suit une routine simple : éditer les fichiers source, compiler l'exécutable et déboguer. Cependant, des erreurs comme oublier de recompiler ou de lier les fichiers peuvent entraîner des pertes de temps. Le programme **make** automatise ces étapes en définissant les relations entre les éléments d'un programme. Il utilise des horodatages (les dates et heures associées aux fichiers) pour déterminer quelles parties du programme ont été modifiées, et donc quelles étapes doivent être refaites. Généralement, la spécification que **make** utilise est enregistrée dans un fichier nommé *Makefile* (ou bien *makefile*) qui comporte un ensemble de règles.

- **Commande générale** : `make [options] [targets]`

- **Options** :

Option	Commentaire
<code>-f filename</code>	Indique dans quel fichier se trouvent les règles, si ce n'est pas <i>Makefile</i> ou <i>makefile</i> .
<code>-C dir</code>	Change de répertoire avant de lire le fichier.
<code>-I dir</code>	Indique où chercher les autres makefiles si l'on a un gros projet.
<code>-n</code>	Affiche les commandes qu'il devrait exécuter sans les exécuter.
<code>-v</code>	Affiche la version de make .

- Une **cible** (ou *target*) est une règle qui définit un fichier à générer ou une action à exécuter, et elle est suivie des prérequis et des commandes à exécuter pour la créer.

- Après l'exécution de la commande **make** dans un shell, il lit le fichier *Makefile* et construit la première cible sur laquelle il tombe (souvent nommée **all**), qui produit un exécutable en règle générale.

Remarque 1 : On peut aussi spécifier une cible après **make** pour ne pas choisir la première, comme **make clean**, par exemple.

Remarque 2 : Comme la cible d'une règle peut être référencée comme le prérequis d'une autre, il est possible de construire un **graphe de dépendances**.

2 Structure d'un *Makefile*

2.1 Les commentaires

Il est possible de faire des commentaires sur une seule ligne avec **#**, équivalent à **//** en C. Or, il est impossible de faire des commentaires sur plusieurs lignes comme en C avec **/*...*/**.

2.2 Les variables

- Les variables stockent toutes une chaîne de caractères. Il n'y a donc pas d'opérations arithmétiques, booléennes, etc.
- Le nom de la variable peut contenir n'importe quel caractère, comme des tirets, commencer par un chiffre, etc., à l'exception d'une virgule (,), du signe égale (=) et d'un dièse (#).

- Pour récupérer la valeur d'une variable, on fait : `$(NOM_VARIABLE)`.

Remarques :

- Si le nom de la variable se compose d'un seul caractère, les parenthèses sont inutiles.
- Pour avoir littéralement la chaîne de caractères `$(NOM_VARIABLE)`, on tapera : `$$$(NOM_VARIABLE)`.

- Il existe une ancienne notation avec des accolades, plus trop utilisée aujourd’hui : `${NOM_VARIABLE}`.
- Il est possible de demander la valeur d’une variable qui n’existe même pas, la valeur retournée sera donc une chaîne de caractères vide.

- Une **variable à expansion simple**, définie par `:=`, est une variable dont la valeur est immédiatement calculée et enregistrée dès que la ligne du *Makefile* est lue.
- Une **variable à expansion récursive**, définie par `=`, est calculée uniquement lorsque la variable est utilisée, équivalente à une variable paresseuse en OCaml.
- L’**affectation conditionnelle** `?=` est réalisée uniquement si la variable n’a pas déjà été initialisée (utile pour les variables d’environnement).
- `+=` permet d’ajouter du texte à une variable et est équivalent à : `var := $(var) new-stuff`.
- Les **wildcards (ou globbing)** représentent un système permettant d’écrire des expressions régulières version éco+. Il est identique à celui du Bourne Shell (bsh) avec les symboles : `~, *, ?, [...], [^...]`.

Exemple	Signification
<code>[a-z]</code>	Toutes les lettres minuscules
<code>*</code>	N’importe quelle séquence de lettres
<code>?</code>	N’importe quelle lettre
<code>ab?</code>	Mot de 3 lettres qui commence par <code>ab</code>
<code>z*.*a</code>	Tous les mots qui contiennent un point, commencent par un <code>z</code> et se terminent par un <code>a</code>
<code>[^A-C]</code>	Toutes les lettres sauf <code>A</code> , <code>B</code> et <code>C</code> (majuscules uniquement)
<code>~</code>	Répertoire personnel (home directory)

- **Variables automatiques :**

<code>\$@</code>	Nom de la cible
<code>\$\$</code>	Nom de l’objet d’une cible d’archive
<code>\$<</code>	Nom du premier pré-requis
<code> \$?</code>	Tous les pré-requis plus récents que la cible, séparés par un espace
<code> \$^</code>	Tous les pré-requis séparés par un espace (sans doublons)
<code> \$+</code>	Tous les pré-requis séparés par un espace (avec doublons), utile pour les références circulaires

- Il existe aussi des variables standards comme `CURDIR` qui contient le répertoire courant (obtenable aussi avec la commande `pwd`).
- Pour ajouter une de nos variables aux variables d’environnement, on utilise `export <VARIABLE>`.

```

1  NAME := Thomas # NAME = 'Thomas'
2
3  # On stocke dans SRC tous les fichiers .c du répertoire courant
4  SRC = $(wildcard *.c) # SRC = 'main.c game.c sound.c gui.c controller.c'
5
6  # On remplace l'extension .c par .o pour tous les fichiers récupérés précédemment
7  OBJ = $(SRC:.c=.o) # OBJ = 'main.o game.o sound.o gui.o controller.o'
8  # Similaire à : OBJ = $(patsubst %.c,%.o,$(wildcard *.c))
9
10 # Comme NAME est déjà défini à 'Thomas', l'affectation ne fait rien
11 NAME      ?= Toto # NAME = 'Thomas'
12 # NICKNAME n'a pas encore été initialisé, donc
13 NICKNAME ?= Toto # NICKNAME = 'Toto'
14
15 FRUITS := Apple
16 FRUITS += Banana Cherish
17 # FRUITS = 'Apple Banana Cherish'
18 # avec un espace entre 'Apple' et 'Banana Cherish'

```

2.3 Les règles

Avant d'exécuter les commandes d'une règle, `make` ne vérifie que l'horodatage. Si une date du pré-requis est plus récente que celle de la cible, alors on exécute les commandes. Si elles sont égales, il n'y a rien à mettre à jour. Si le pré-requis est inexistant, `make` va d'abord chercher une règle du même nom, puis vérifier si le fichier en question existe.

```
1  # Forme générale
2  target1 target2 target3: prerequisite1 prerequisite2
3      commande1 # Ne pas oublier la tabulation devant la commande !
4      commande2
5      commande3 \ # On peut découper une commande sur plusieurs lignes
6      suite_commande3 \
7      fin_commande3
8      commande4
9
10 # Règle explicite
11 file.o: file.c file.h
12     @gcc -c file.c
13 # Ici la commande "gcc -c file.c" ne sera pas affichée dans le shell à cause de @
14
15 # Règle implicite
16 # make a des règles prédéfinies pour générer des .o
17 # à partir des fichiers vpath.c et variable.c
18 # Si un des fichiers .h est modifié alors les fichiers vpath.o et variable.o
19 # seront recompilés
20 vpath.o variable.o: make.h config.h getopt.h gettext.h dep.h
21
22 # Règle par motifs (pattern)
23 %.class: %.java
24     javac $<
25 # La commande : make hello.class
26 # exécutera la commande : javac hello.java
```

- On peut simplifier un **Makefile** comme suit grâce aux règles par motifs déclarées implicitement.

```
1  VPATH = src include
2  CPPFLAGS = -I include
3  count_words: counter.o lexer.o -lfl
4  count_words.o: counter.h
5  counter.o: counter.h lexer.h
6  lexer.o: lexer.h
7
8  %.o: %.c
9      $(COMPILE.c) $(OUTPUT_OPTION) $<
10 # aussi équivalent au double-suffix rule
11 .c.o:
12     $(COMPILE.c) $(OUTPUT_OPTION) $<
13
14 %.c: %.l
15     @$(RM) $@
16     $(LEX.l) $< > $@
17
18 %: %.c
19     $(LINK.c) $^ $(LOADLIBES) $(LDLIBS) -o $@
20 # équivalent au single suffix rule
21 .c:
22     $(LINK.c) $^ $(LOADLIBES) $(LDLIBS) -o $@
23
24 # Les suffixes par défaut sont :
```

```

25 .SUFFIXES: .out .a .ln .o .c .cc .C .cpp .p .f .F .r .y .l
26
27 # pour appliquer la règle qu'à certains fichiers
28 # On parle de static pattern rules
29 $(OBJECTS): %.o: %.c
30     $(CC) -c $(CFLAGS) $< -o $@
31
32 une_ligne: ; @echo Hello ; echo World !

```

- Les **modificateurs de commande** qui doivent être placés en préfixe sont :

Préfixe	Commentaire
@	Ne pas afficher la commande (pour rendre la sortie de make plus agréable à lire).
-	Indique d'ignorer les erreurs de la commande. Par défaut, s'il y a une erreur, make s'arrête.
+	Exécute l'instruction même si l'option -n est utilisée.

- Une **phony target** est une cible qui permet d'exécuter un script (et non de compiler un fichier). Elle est donc toujours exécutée puisque la commande ne crée jamais le fichier cible. Or, **make** ne fait pas la différence entre une target et une phony target, il faut donc spécifier à **make** que **clean** ne doit pas être considéré comme un fichier à créer mais comme une commande avec **.PHONY**. Même si le fichier **clean** existe, la commande sera exécutée.

```

1 .PHONY: clean all # etc.
2
3 clean:
4     rm -rf *.o
5     # Cette règle permet de supprimer tous les fichiers .o
6     # La commande rm ne peut pas échouer grâce à l'option -f

```

Remarque : Il existe d'autres cibles spéciales pour modifier le comportement de **make** par défaut, comme **.INTERMEDIATE**, **.SECONDARY**, **.PRECIOUS**, **.DELETE_ON_ERROR**.

- **VPATH** permet d'indiquer dans quels dossiers regarder pour chercher les fichiers sources (en plus du répertoire courant où se trouve le fichier **Makefile**).

```

1 VPATH = src
2 CPPFLAGS = -I include
3 # ou mieux
4
5 # indique de chercher tous les fichiers .c dans le dossier src
6 vpath %.c src
7
8 # indique de chercher tous les fichiers .h dans le dossier include
9 vpath %.h include

```

- Les **double-colon rules** permettent de surcharger une règle :

```

1 # 1
2 file-list:: generate-list-script
3     chmod +x $<
4     generate-list-script $(files) > file-list
5 # 2
6 file-list:: $(files)
7     generate-list-script $(files) > file-list
8
9 # Si generate-list-script a été modifié alors on exécute 1
10 # Si les fichiers sources ont été modifiés alors on exécute 2

```

2.4 Autres concepts

- Il est possible d'inclure un autre Makefile comportant des définitions de variables/macros et de règles dans notre propre Makefile avec `include <FILE>`.
- Pour de gros projets, il faut parfois un *Makefile* par dossier. Il faudra donc utiliser le concept de `make recursive` avec la commande : `$(MAKE) --directory=$@` à utiliser dans le Makefile principal pour exécuter les Makefiles dans les sous-dossiers.

```
1  # Exemple de conditions
2  ifdef COMSPEC
3      PATH_SEP := ;
4      EXE_EXT := .exe
5  else
6      PATH_SEP := :
7      EXE_EXT :=
8  endif
9
10 ifeq "a" "a"
11 # ou bien ifeq 'a' 'a' # ou encore (a, a)
12 # ...
13 endif
14
15 # Exemple de boucle
16 SUBDIRS = foo bar baz
17
18 subdirs:
19     for dir in $(SUBDIRS); do \
20         $(MAKE) -C $$dir; \
21     done
22
23 # Pour générer des fichiers de dépendances :
24 %.d: %.c
25     @set -e; rm -f $@; \
26     $(CC) -M $(CPPFLAGS) $< > $@.$$$$; \
27     sed 's,\($*\)\.o[ :]*,\1.o $@ : ,g' < $@.$$$$ > $@; \
28     rm -f $@.$$$$
29
30 include $(sources:.c=.d)
31
32 # Appeler une fonction
33 $(function arguments)
34
35 # Exemples
36 comma := ,
37 empty :=
38 space := $(empty) $(empty)
39 foo := a b c
40 bar := $(subst $(space),$(comma),$(foo))
41 # bar = 'a,b,c'.
42
43 # Supprime les espaces de début et de fin de la chaîne
44 # et remplace chaque séquence interne d'un ou plusieurs caractères d'espacement par un seul espace.
45 $(strip a b c ) # donne 'a b c'.
46
47 # Pour réaliser un filtrage
48 sources := foo.c bar.c baz.s ugh.h
49 $(filter %.c %.s,$(sources)) # donne 'foo.c bar.c baz.s'
50
51 # Il existe beaucoup d'autres fonctions : notdir, basename, etc. que je vous laisse découvrir
```

3 Rappels sur GCC

- **Définition :** **GCC** (*GNU C Compiler*) est un compilateur portable pour le langage C créé par Richard Stallman. Il transforme le code source en langage machine pour créer un fichier exécutable en passant par des étapes de prétraitement, compilation, assemblage et édition de liens.
- **Commande générale :**

```
1 gcc [-c|-S|-E] [-std=standard] [-g] [-pg] [-Olevel] [-W...] [-pedantic] [-Idir...] [-Ldir...]
2 [-Dmacro[=defn]...] [-Umacro] [-foption...] [-mmachine-option...] [-o outfile] [@file]
```

Remarque : Généralement, l'ordre des options n'a pas trop d'importance. Mais attention, mettez bien vos bibliothèques après les fichiers sources.

- **Options :**

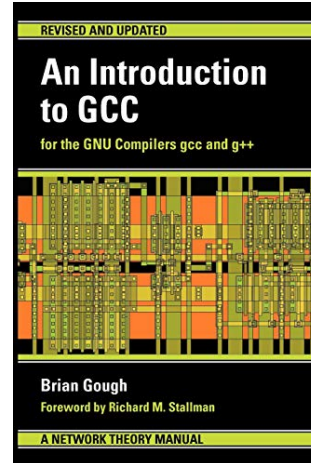
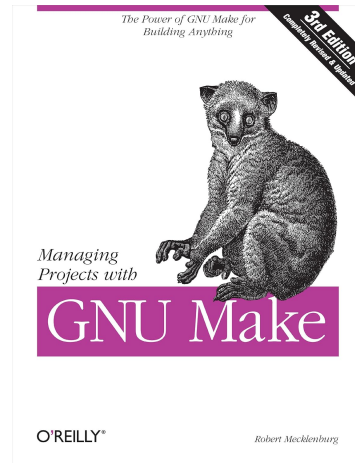
Option	Commentaire
-c	N'exécute pas l'éditeur de liens. La sortie se compose de fichiers objets (*.o) produits par l'assembleur.
-S	N'exécute pas l'assembleur. La sortie se compose de fichiers en langage d'assemblage (*.s) produits par la compilation.
-E	N'exécute pas la compilation. La sortie se compose de fichiers (*.i) envoyés vers la sortie standard, produits par le prétraitement.
-g[level]	Ajoute des informations de débogage (pour gdb). Par défaut, level=2 mais il varie entre 0 et 3.
-pg	Ajoute des informations de profilage adaptées au programme d'analyse gprof .
-O<level>	Optimise le code. Les niveaux sont : 0, 1, 2, 3, fast et s (pour size, la taille de l'exécutable).
-Werror	Transforme tous les avertissements en erreurs.
-Wall	Active les avertissements courants.
-Wextra	Active des avertissements supplémentaires.
-pedantic	Renvoie des avertissements si les normes ISO ne sont pas respectées.
-pedantic-errors	Renvoie des erreurs si les normes ISO ne sont pas respectées.
-I<path>	Spécifie un chemin où trouver les en-têtes *.h.
-L<path>	Spécifie un chemin où trouver les bibliothèques statiques *.a.
-l<name>	Donne la bibliothèque statique lib<name>.a à inclure dans notre exécutable.
-Dmacro[=value]	Crée une macro comme #define macro value en C.
-Umacro	Supprime la macro comme #undef macro en C.
-v	Active le mode verbose pour donner plus d'informations sur la compilation.
-o filename	Spécifie le nom de fichier en sortie, que ce soit l'exécutable, un fichier objet, une archive, etc.
-s	Supprime toutes les tables de symboles et les informations de relocation de l'exécutable. Utile pour rendre plus difficile le reverse engineering sur son programme.

Remarque : Si votre fichier source contient des portions de code incorrectes, GCC vous les signalera soit par une erreur, soit par un avertissement en indiquant : **fichier:n°ligne:erreur**. Contrairement à un avertissement, une erreur stoppe la compilation et ne produit rien en sortie.

- **Exemples :**

- Si j'ai les fichiers *main.c*, *game.c*, *sound.c*, *game.h* et *sound.h* tous situés dans le même dossier et que je souhaite pouvoir déboguer avec gdb et m'assurer de faire le moins de fautes possibles, alors pour compiler mon exécutable je ferai :
`gcc -Wall -Wextra -Werror -g main.c game.c sound.c -o fortnite.exe.`
- Si je souhaite juste produire *game.o* à partir de *game.c* qui utilise des fonctions de la bibliothèque **math**, alors la commande sera : `gcc -c game.c -lm -o game.o.` (Attention : c'est bien **-lm** et non **-lmath**).
- Pour créer une bibliothèque statique *libgame.a* à partir des fichiers objets *sound.o*, *controller.o*, et *gui.o*, on utilisera la commande :
`ar rcs libgame.a sound.o controller.o gui.o.`
Pour l'utiliser, on utilisera l'option **-lgame**.

4 Références



- Pour plus de détails, consultez le manuel officiel de `make` et GCC disponible aux adresses suivantes : [*Manuel de GNU make*](#) et [*Manuel de GCC*](#).